

Storage Classes

Storage Classes	Storage Place	Default Value	Scope	Lifetime
auto	RAM	Garbage Value	Local	Within function/block
extern	RAM	Zero	Global	Till the end of the main program Maybe declared anywhere in the program
static	RAM	Zero	Local	Within the function till the end of the main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within the function

Storage Classes

- ▶ `#include<stdio.h>`
- ▶ `int e;`
- ▶ `void main()`
- ▶ `{`
- ▶ `auto int a;`
- ▶ `extern int e;`
- ▶ `static int s;`
- ▶ `register int r;`
- ▶ `printf("\n %d -- %d -- %d -- %d",a,e,s,r);`
- ▶ `}`

- ▶ `// 0 -- 0 -- 0 -- 1`

Dynamic Memory Allocation



Dynamic Memory Allocation

- ▶ `malloc()`
 - ▶ `calloc()`
 - ▶ `free()`
 - ▶ `realloc()`
- ▶ These functions are in `<stdlib.h>` library

malloc()

- ▶ “**malloc**” or “**memory allocation**” method is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form.

malloc()

- ▶ **Syntax:**
- ▶ **`ptr = (cast-type*) malloc(byte-size)`** For
- ▶ **Example: `ptr = (int*) malloc(100 * sizeof(int));`**
- ▶ Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.

```
▶ int main()
▶ {
▶
▶     int* ptr;
▶     int n, i, sum = 0;
▶     n = 5;
▶     printf("Enter number of elements: %d\n", n);
▶
▶     // Dynamically allocate memory using malloc()
▶     ptr = (int*)malloc(n * sizeof(int));
▶
▶     // Check if the memory has been successfully
▶     // allocated by malloc or not
▶     if (ptr == NULL) {
▶         printf("Memory not allocated.\n");
▶         exit(0);
▶     }
▶ }
```

```
▶ else {  
▶  
▶     // Memory has been successfully allocated  
▶     printf("Memory successfully allocated using  
malloc.\n");  
▶  
▶     // Get the elements of the array  
▶     for (i = 0; i < n; ++i) {  
▶         ptr[i] = i + 1;  
▶     }  
▶  
▶     // Print the elements of the array  
▶     printf("The elements of the array are: ");  
▶     for (i = 0; i < n; ++i) {  
▶         printf("%d, ", ptr[i]);  
▶     }  
▶ }  
▶  
▶ return 0;  
▶ }
```

- ▶ OUTPUT :
- ▶ Enter number of elements: 5 Memory successfully allocated using malloc. The elements of the array are: 1, 2, 3, 4, 5

calloc()

- ▶ “**calloc**” or “**contiguous allocation**” method is used to dynamically allocate the specified number of blocks of memory of the specified type. It initializes each block with a default value ‘0’.

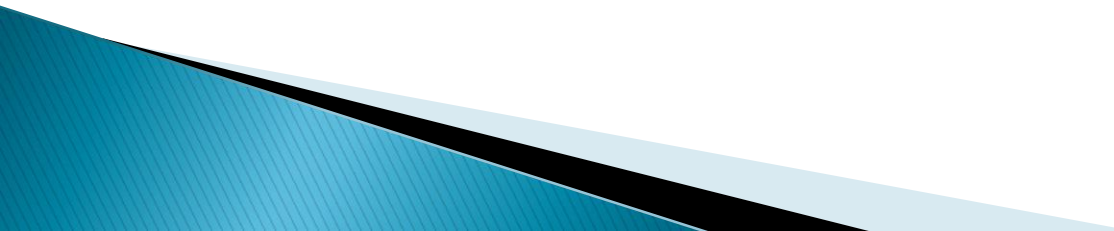
calloc()

- ▶ **Syntax:**
- ▶ **`ptr = (cast-type*)calloc(n, element-size);`** For
- ▶ **Example: `ptr = (float*) calloc(25, sizeof(float));`**
- ▶ This statement allocates contiguous space in memory for 25 elements each with the size of float.

free()

- ▶ “**free**” method is used to dynamically **de-allocate** the memory. The memory allocated using functions `malloc()` and `calloc()` are not de-allocated on their own. Hence the `free()` method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.
- ▶ **Syntax :** `free(ptr);`

realloc()

- ▶ **“realloc” or “re-allocation”** method is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.
- 

realloc()

- ▶ **Syntax:**
- ▶ **`ptr = realloc(ptr, newSize);`**
- ▶ where ptr is reallocated with new size 'newSize'.
- ▶ Example : `ptr = realloc(ptr, n * sizeof(int));`

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *arr = (int*) malloc(5 *
sizeof(int));
    int i;

    if (arr == NULL) {
        printf("Memory allocation
failed.\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }

    if (arr == NULL) {
        printf("Memory reallocation
failed.\n");
        return 1;
    }
}
```

```
printf("Array before resizing:\n");
for (i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

arr = (int*) realloc(arr, 10 *
sizeof(int));

for (i = 5; i < 10; i++) {
    arr[i] = i + 1;
}

printf("Array after resizing:\n");
for (i = 0; i < 10; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

free(arr);

return 0;
}
```